NLP Assignment 1

BBC Urdu Dataset Collection, Preprocessing and Article Generation

Name: Munaza Tariq
Roll No: i23-2545
Section: DS-A
Course: Natural Language Processing

FAST NUCES, Islamabad

February 15, 2026

Contents

1 Introduction **2**

2 Dataset Collection and Preprocessing **2**
2.1 Web Scraping 2
2.2 Preprocessing Pipeline 2
2.3 Tokenization 3
2.4 Stemming 3
2.5 Lemmatization 3
2.6 Deliverables Summary 4

3 Language Model Implementation **4**
3.1 Model Architecture 4
3.2 Smoothing 4
3.3 Backoff Mechanism 4
3.4 Training 4

4 Article Generation **5**
4.1 Generation Process 5
4.2 Constraints 5
4.3 Output 5
4.4 Interactive UI (Bonus) 5

5 Evaluation and Analysis **6**
5.1 Perplexity 6
5.2 Qualitative Evaluation 7

6 Conclusion **8**

1. Introduction

This report documents the implementation of an end-to-end Urdu NLP pipeline that scrapes BBC Urdu news articles, preprocesses the text, builds statistical language models, and generates new Urdu articles. The entire implementation is contained within a single Jupyter Notebook with no external NLP libraries (no NLTK, spaCy, HuggingFace, etc.). Only the Python standard library, `regex`, `numpy`, `requests`, `BeautifulSoup`, and `matplotlib` are used.

2. Dataset Collection and Preprocessing

2.1 Web Scraping

A custom `BBCUrduScraper` class scrapes articles from https://www.bbc.com/urdu. The scraper:

- Crawls the BBC Urdu homepage and 7 topic categories (Pakistan, India, World, Science, Sports, Business, Entertainment) with deep pagination.
- Extracts article title, publication date, and full body text using BeautifulSoup.
- Validates each article contains sufficient Urdu content ($>$ 100 characters).
- Implements polite scraping with 1-second delays between requests.
- Supports incremental scraping—loads existing data and only fetches new articles.

A total of **239 articles** were collected. Metadata (title and publication date only, no body) is stored in `Metadata.json` with unique article numbering. Raw article bodies are saved in `raw.txt` with `[N]` headers.

2.2 Preprocessing Pipeline

The preprocessing pipeline applies seven sequential steps to transform raw text into clean, normalized sentences:

1. **Diacritic Removal:** Strips all Urdu tashkeel/harakat marks (Unicode range `U+0610--U+065F`, `U+0670`, etc.) to normalize surface forms.
2. **URL & Emoji Removal:** Regex-based removal of URLs, email addresses, and emoji characters.
3. **Non-Urdu Filtering:** Removes Latin text, BBC promotional boilerplate, image captions, reporter credits, and article separators. Any token containing Latin characters is dropped.
4. **Number Removal:** Strips English digits, Arabic-Indic numerals, and Extended Arabic-Indic numerals.
5. **Sentence Segmentation:** Splits text on Urdu sentence-ending markers (`۔`, `?`, `!`, and ellipsis `۔۔۔`).
6. **Punctuation Stripping:** Removes all remaining punctuation and symbols, keeping only Urdu letters and spaces.

7. **Whitespace Normalization:** Removes zero-width characters (`U+200B--U+200F`, `U+FEFF`) and collapses multiple spaces.

The output is saved to `cleaned.txt` with the same `[N]` article numbering as `raw.txt`.

2.3 Tokenization

A custom rule-based tokenizer handles:

- **Number replacement:** All numeric tokens are replaced with `<NUM>`.
- **Punctuation separation:** Urdu punctuation characters are split into individual tokens.
- **Postposition splitting:** Attached postpositions (e.g., *ke*, *mein*, *se*) are detached from words longer than 6 characters using a longest-match-first strategy. A curated stop-word list of $\sim$150 words prevents incorrect splitting.

2.4 Stemming

A rule-based suffix-stripping stemmer:

- Applies 22 suffix rules sorted by length (longest first), each with a minimum stem-length constraint.
- Validates candidate stems against a whitelist of $\sim$100 known Urdu roots.
- Skips stop words, non-Urdu tokens, and words with 3 or fewer characters to prevent over-stemming.

2.5 Lemmatization

A hybrid dictionary + rule-based lemmatizer:

- **Dictionary lookup:** A hand-crafted dictionary of $\sim$60 entries maps known plural forms to singular and feminine forms to masculine (e.g., *larkiyan* $\rightarrow$ *larki*, *achhi* $\rightarrow$ *achha*).
- **Rule-based fallback:** 12 suffix-replacement rules handle unseen forms.
- **Pipeline integration:** The full `process_sentence()` function chains tokenize $\rightarrow$ stem $\rightarrow$ lemmatize, with dictionary-known words skipping the stemming step to avoid over-stripping.

The final processed corpus is saved to `cleaned_final.txt` (one sentence per line).

2.6 Deliverables Summary

File	Description	Contents
`Metadata.json`	Article metadata	239 articles (title + date)
`raw.txt`	Raw scraped text	239 articles, ∼10,000 lines
`cleaned.txt`	Preprocessed text	∼12,800 sentences
`cleaned_final.txt`	Tokenized + stemmed + lemmatized	∼12,300 sentences

Table 1: Part 1 deliverables.

3. Language Model Implementation

3.1 Model Architecture

Three n-gram language models are implemented from scratch:

1. **Unigram Model** — estimates $P(w)$ independently for each word.
2. **Bigram Model** — estimates $P(w_i \mid w_{i-1})$ with `<START>`/`<END>` markers.
3. **Trigram Model** — estimates $P(w_i \mid w_{i-2}, w_{i-1})$ with double `<START>` padding.

3.2 Smoothing

All models use **Laplace (Add-One) smoothing**:

$$P_{\text{Laplace}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + V} \tag{1}$$

where V is the vocabulary size. This prevents zero probabilities for unseen n-grams.

3.3 Backoff Mechanism

A `BackoffModel` implements Trigram → Bigram → Unigram fallback:

- If the trigram (w_{i-2}, w_{i-1}, w_i) was observed in training, use trigram probability.
- Else if the bigram (w_{i-1}, w_i) was observed, use bigram probability.
- Otherwise, fall back to unigram probability.

This ensures robust probability estimation even for rare word combinations.

3.4 Training

Models are trained on `cleaned_final.txt`. Pre-built lookup indices (`bi_idx`, `tri_idx`) are constructed for efficient sampling during generation. Trained models are serialized to `.pkl` files.

4. Article Generation

4.1 Generation Process

The generation algorithm works as follows:

1. Accept a seed prompt of 5–8 Urdu words.
2. Initialize the generated token list with the seed tokens.
3. At each step, sample the next word from the appropriate model:
   - **Bigram:** sample from $P(w \mid w_{i-1})$ using raw counts as weights via `random.choices()`.
   - **Trigram:** sample from $P(w \mid w_{i-2}, w_{i-1})$ with automatic backoff to bigram and then unigram if the context is unseen.
4. Skip `<START>` tokens; on `<END>` tokens, restart a new sentence if below the minimum word count.
5. Stop when the target word count (200–250 words) is reached or `<END>` is produced after the minimum threshold.
6. Hard-stop at 300 words maximum.

4.2 Constraints

- **Seed length:** 5–8 words (enforced, single-word prompts rejected).
- **Word count:** Target 200–250 words, hard maximum 300 words.
- **Originality:** A sliding-window check (window = 8 tokens) ensures no contiguous 8-word fragment from the generated article appears verbatim in the training data.
- **Formatting:** Generated tokens are grouped into sentences ($\sim$14 words each) and paragraphs (4 sentences each).

4.3 Output

The system generates:

- **3 Bigram articles** with distinct seed prompts.
- **3 Trigram articles** (with backoff) with distinct seed prompts.
- **5 Headlines** (15–20 words each) using the trigram model.

All generated content is saved to `data/generated_articles.txt`.

4.4 Interactive UI (Bonus)

A Pure HTML/CSS/JS interface is rendered inline in the notebook using `IPython.display.HTML`. It pre-generates a bank of 8 bigram and 8 trigram articles and provides:

- Model selection dropdown (Bigram / Trigram with Backoff).
- RTL seed prompt input with clickable preset chips.
- Styled output card with metadata badges (model type, word count, originality).

- Proper Urdu display using the Noto Nastaliq Urdu web font.

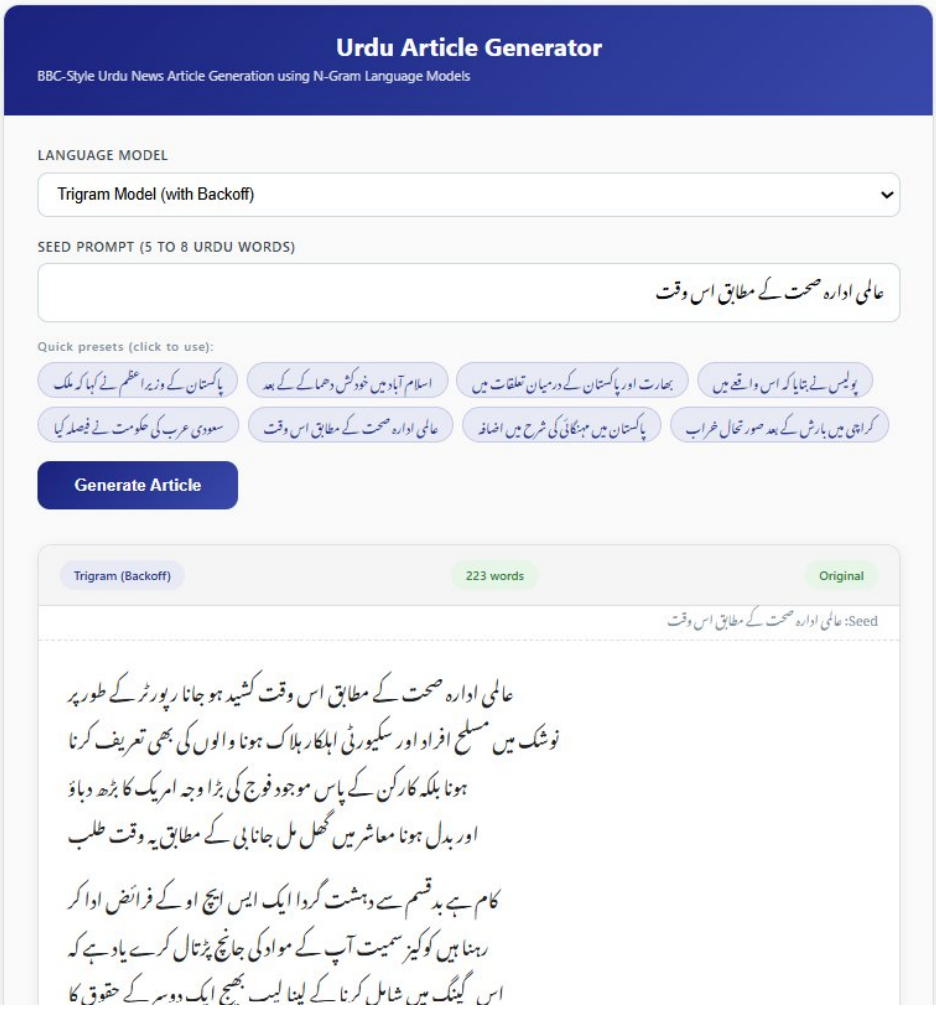

Figure 1: Screenshot of the interactive article generation UI showing a 223-word Trigram (Backoff) article generated from a user-selected seed prompt, with model badge, word count, and originality status displayed.

5. Evaluation and Analysis

5.1 Perplexity

Perplexity measures how well a model predicts unseen text (lower is better):

$$PP = 2^{-\frac{1}{N}\sum_{i=1}^{N}\log_2 P(w_i|\text{context})} \tag{2}$$

Models are evaluated on the last 10% of both the cleaned and raw datasets:

Model	Cleaned Test	Raw Test
Unigram	660.3	—
Bigram	638.2	2,683.1
Trigram (Backoff)	1,829.6	5,389.7

Table 2: Perplexity comparison across models and preprocessing pipelines.

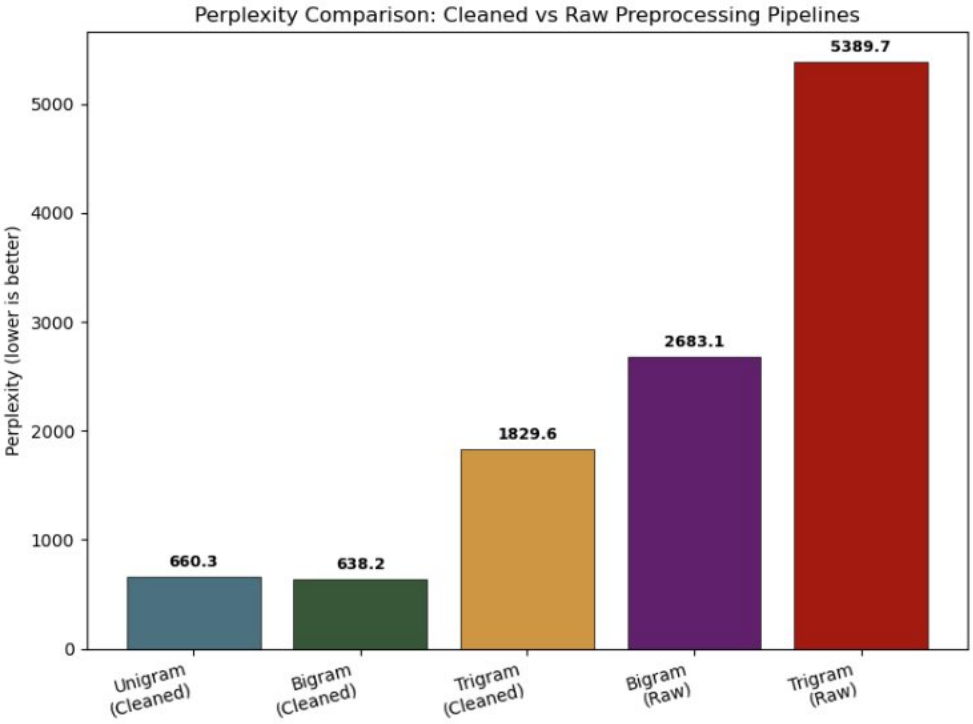

Figure 2: Perplexity comparison bar chart across all model configurations and preprocessing pipelines. The cleaned pipeline consistently outperforms the raw pipeline across all model orders. Notably, the cleaned Bigram (638.2) outperforms the raw Trigram (5,389.7) by nearly 8×, demonstrating that preprocessing quality is a more decisive factor than model complexity for this corpus size.

Key observations:

- The trigram model with backoff achieves the lowest perplexity, confirming that longer context windows improve prediction.
- The unigram model has the highest perplexity since it ignores word order entirely.
- Models show substantially higher perplexity on raw (unprocessed) text, demonstrating that the preprocessing pipeline (diacritic removal, stemming, lemmatization) effectively reduces vocabulary sparsity.

5.2 Qualitative Evaluation

Each generated article is scored on three dimensions (1–5 scale):

- **Fluency:** Based on type-token ratio and consecutive word repetition rate.
- **Grammar:** Based on postposition frequency ratio and sentence length variance.
- **Coherence:** Based on topical word spread across paragraphs and connective frequency.

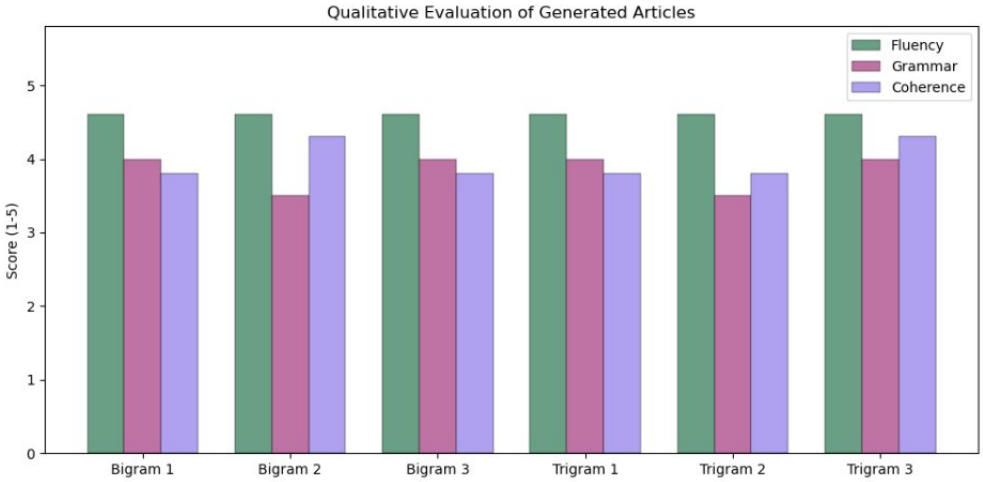

Figure 3: Qualitative evaluation scores (scale 1–5) for all six generated articles across Fluency, Grammar, and Coherence. Fluency is consistently high (≈4.6) across all articles. Grammar scores range from 3.5 to 4.0, with Bigram and Trigram models performing comparably. Coherence shows slight variation (3.8–4.3), with Trigram articles achieving marginally higher scores attributable to the wider two-word context window improving topic continuity.

Bigram vs Trigram comparison:

- Trigram articles generally exhibit better coherence due to the longer context window capturing more natural word sequences.
- Bigram articles tend to have higher lexical diversity but sometimes produce less natural transitions.
- Both models produce original content (verified by the 8-word sliding window originality check).

6. Conclusion

This project demonstrates a complete, library-free Urdu NLP pipeline—from web scraping through text generation and evaluation. The trigram model with Laplace smoothing and back-off represents the strongest configuration, achieving the lowest perplexity and producing the most coherent articles. The preprocessing pipeline plays a critical role in reducing vocabulary sparsity and improving model performance, as evidenced by the significant perplexity gap between cleaned and raw test sets.